TRANSFORMED 2 SUCCEED

transformed2succeed.com

FREE LIVE SESSION · 30 MINUTES

Linux for AI-ML Systems

Every model is a file on a Linux machine. Learn to read the machine, feed the model, and keep it running.

PRESENTER Dr. Emmanuel Naweji **LIVE** Mondays & Thursdays · 7:30 PM Central · 30 min

TONIGHT'S 30 MINUTES

Welcome! Here's the plan

0–2 min

Welcome

Say hi in the chat: your name, where you're joining from, and your role.

2–20 min

Three modules

One real idea from each module of Linux for AI-ML Systems, with the lab behind it.

20–23 min

Keep going

How to get the full course, every lab, and support along the way.

23–30 min

Your questions

Live Q&A. Drop questions in the chat any time; I'll answer at the end.

WHY THIS COURSE

Your model is a file and a process. Learn to read both.

01

Find the file

A model is a big file in a folder. Find it, measure it, and prove what's wasting disk.

02

Feed it safely

Before any long job, check space, owner and secret, so it never dies at 97%.

03

Keep it alive

When a model process dies, Linux already wrote down why. Read the right log first.

THE COURSE MAP

Three modules, three phrases

MODULE 1

The Machine Under Your Model

File at rest, process at work. Diagnose a server you've never seen.

MODULE 2

Feed the Model

Space · Owner · Secret: the pre-flight that saves your GPU hours.

MODULE 3

Keep It Running

Don't guess. Grep. Catch a silent out-of-memory kill and fix it.

01

MODULE 1 OF 3

The Machine Under Your Model

A model is a file at rest and a process at work. Learn to read both.

A model is a file you can measure

8B

parameters in a model like Llama 3 8B

× 2 B

bytes per parameter at 16-bit precision

≈ 16 GB

one file on disk

≈ 5 GB

same model, squeezed to 4-bit

That file lives on a Linux machine, in a folder, owned by a user, read by a process.

File at rest. Process at work.

FILE AT REST

Bytes on disk, waiting

Wrong path, missing file, full disk, no permission to read it. Tools: find, du, ls.

VS

PROCESS AT WORK

Loaded in memory, with a PID

Crashed, frozen, out of memory, on the wrong GPU. Tools: ps, top, kill.

First question on any broken server: file problem, or process problem? Answer it and you cut the search in half.

Linux is a house. Every room has one job.

01 **/ the front door**

The root. Every path starts here.

02 **/home your desk**

Your files. ~ is the shortcut. Hugging Face caches models here.

03 **/opt the workshop**

Installed apps. Often /opt/models too.

04 **/var/log the diary**

Logs: what happened, and when.

05 **/dev the devices**

Disks, and GPUs as /dev/nvidia0.

06 **/proc the live feed**

Real-time facts on every running process.

A command is a sentence: ls -lh models/ = verb, how, what. pwd tells you where you're standing.

Small tools, joined by pipes

TERMINAL

```
$ du -sh * | sort -h
$ du -ah models | sort -rh | head -5
```

`du -sh / du -ah`
disk usage: one total per item / every file

`sort -h / sort -rh`
sort human sizes; -r puts the biggest first

`head -5`
keep only the top five lines

`|`
the pipe: hands one tool's output to the next

YOU SHOULD SEE 39M models, then 17M models/old/llama-3-8b-backup, the same size as the real model. Suspicious.

Lab: map a server you've never seen

1

Mission 1 · Disk

What is eating the disk? du and sort point at models/old.

2

Mission 2 · Startup

Why won't the LLM start? grep -rn ERROR logs finds llama3-8b vs llama-3-8b.

3

Mission 3 · Duplicate

Which file is wasted? sha256sum proves two identical fingerprints.

4

Fix, then score

sed fixes the typo, rm -r removes the copy, check_answers.sh: 3 / 3.



Measure, prove, then act.

rm has no undo and no trash can. du, grep and sha256sum come before you move, fix or delete anything.

02

MODULE 2 OF 3

Feed the Model

A model can only use what it can reach. Check space, owner and secret first.

The job that died at 97%

Hour 1

training starts; loss falls, GPU busy, all green

97%

the final checkpoint save begins

0 bytes

left on the disk

Errno 28

No space left on device

The code was fine. The GPU was fine. The disk was full of old checkpoints nobody deleted.

Pre-flight: Space · Owner · Secret

01

Space

Room for checkpoints, logs and outputs? df -h, df -i, du -sh * | sort -h.

02

Owner

Can the job's user read the data? ls -l, then chmod 640. Never chmod 777.

03

Secret

Token loaded from the environment, file private? source .env, chmod 600.

Pilots don't skip the checklist because they flew yesterday. Do the math: checkpoint size × kept + 20%.

Same key. Two very different risks.

IN THE CODE

token = "hf_AbC123..."

Ends up in git history, screenshots and chat. Anyone with the repo has your key.

VS

IN THE ENVIRONMENT

export HF_TOKEN=...

Kept in a .env file, chmod 600, listed in .gitignore, loaded with source .env.

Read the ten characters: -rw-r----- = owner rw, group r, others nothing. 640 for data, 600 for secrets.

Lock the secret, load the key

TERMINAL

```
$ chmod 640 data/tickets.jsonl
$ ls -l .env && chmod 600 .env
$ source .env && echo ${LAB_HF_TOKEN:0:6}
```

`chmod 640`
owner read+write, group read, others nothing

`chmod 600`
owner only: the right setting for secrets

`source .env`
runs the export line in this shell

`${NAME:0:6}`
peek at 6 characters. Never echo a whole key

YOU SHOULD SEE hf_lab, then PASS SECRET when you re-run python3 train.py.

Lab: fix a job that won't start

1

Watch it refuse

python3 train.py prints three FAIL lines: space, owner, secret.

2

Free the space

du shows old-run at 81 MB. Keep best/, rm -r the rest. PASS SPACE.

3

Fix owner, then secret

chmod 640 the dataset. chmod 600 .env and source it. PASS, PASS.

4

Run to 100%

Three PASS lines, three epochs, three checkpoints. Training code untouched.

03

MODULE 3 OF 3

Keep It Running

When a model dies, Linux already wrote down why. Don't guess. Grep.

The server vanished. Its log says nothing.

02:14

last normal line in the app log

0

errors in the app log

1 line

in a different log: the kernel's

Killed

Out of memory: Killed process 4121 (python3)

Memory ran out, so Linux killed the biggest process to save the machine. The app never got to say goodbye.

Ask first. Force last.

SIGTERM

kill 4121 · "please stop"

The app can finish requests, save state and close files. Always try this first.

VS

SIGKILL

kill -9 4121 · "stop now"

Instant. No cleanup, no last log line. The OOM killer uses this one.

That's why the silent kill was silent: SIGKILL gave the server no chance to write a single word.

The log ladder: read before you restart

1

1 · The app's own log

tail -n 50 logs/server.log. What did my code say?

2

2 · The kernel's log

sudo dmesg -T | grep -i 'killed process'. Did Linux step in?

3

3 · The service's log

journalctl -u ai-server --since '1 hour ago'. What did systemd see?

Lab: climb the ladder after the kill

TERMINAL

```
$ pgrep -af serve.py || echo 'not running'
$ tail -3 logs/server.log
$ grep -i 'killed process' logs/*.log
```

`pgrep -af NAME`
find processes by name, with the full command

`A || B`
run B only if A failed

`tail -3`
the last three lines: no error here

`grep -i`
search, ignoring upper and lower case

YOU SHOULD SEE not running, a normal last app line, then kernel-sim.log: Out of memory: Killed process (python3).

Lab: catch a silent kill, then fix it

1

Run and measure

python3 serve.py & plus the watchdog. ps shows about 20 MB at rest.

2

Overload it

load.py sends long requests. The cache climbs 32 MB each, then: Killed.

3

Fix the cause

sed sets max_cache_mb=100. Restart. All 10 requests ok, cache stays at 100.

4

Stop it cleanly

Plain kill: 'shutting down cleanly'. kill -9 only if it's stuck.



Weights are fixed. The cache grows. Limit the part that grows.

A restart only resets the clock. A limit is the fix. Restart=on-failure is a safety net, not a cure.

THE WHOLE COURSE

Three modules. Three phrases to keep.

File at rest. Process at work.

Module 1. First question on any broken server: which one is it?

Space · Owner · Secret

Module 2. The pre-flight before any long job or download.

Don't guess. Grep.

Module 3. Read the logs, in order, before you restart.



You can do this.

Map a strange server. Rescue a training job. Solve a silent kill. Three labs, one repo, your own terminal.

THE MEMBERSHIP

Every course. One price.

$49

per month

All 8 courses, every module and every lab, plus a live session with me every month.

ALL 8 COURSES INCLUDED

- Linux for AI-ML Systems
- Python for AI-ML
- Zero to AI-ML Systems Engineering
- DevOps for AI-ML Systems
- SRE for AI-ML Systems
- GPU Optimization for AI-ML Systems
- FinOps for AI-ML Systems
- AI Agent Operations

BUILD WITH ME, LIVE

Once a month, we build something together

1

Learn at your pace

Work through any course, any module, on your own schedule.

2

Bring a real problem

An idea, a project at work, or a lab you want to take further.

3

Meet me live

Once a month we get on a call and build it together, step by step.

4

Leave with it built

Something real you made, understand and can show.

FREE LIVE SESSIONS

Every Monday and Thursday, 7:30 PM Central

A different course each session. The full rotation takes 4 weeks, then starts again.

	WEEK 1	WEEK 2	WEEK 3	WEEK 4
MONDAY	TONIGHT **Linux for AI-ML Systems**	**Zero to AI-ML Systems Engineering**	**SRE for AI-ML Systems**	**FinOps for AI-ML Systems**
THURSDAY	NEXT **Python for AI-ML**	**DevOps for AI-ML Systems**	**GPU Optimization for AI-ML Systems**	**AI Agent Operations**

Join the membership today

$49/month · all 8 courses · a live build session with me every month

transformed2succeed.com

Next free session: Thursday, 7:30 PM Central · Python for AI-ML

Questions? Let's take them now.

Dr. Emmanuel Naweji · Transformed 2 Succeed